

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 – 50 Q2 – 50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Q1. Multimedia Rendering Engine :

A multimedia rendering engine evaluates three algorithms with complexities $O(n)$, $O(n \log n)$, and $O(n^2)$. Analyze each algorithm using asymptotic notation and compare their growth rates. Explain how performance differs when processing high-resolution media files. Identify the most efficient algorithm for large-scale rendering tasks. Justify your answer based on computational efficiency and scalability.

ANSWER:

Introduction

A multimedia rendering engine converts multimedia content such as images, videos, animations, and 3D graphics into the final visual output. As media resolution increases (e.g., 4K, 8K, VR), the choice of algorithm significantly affects rendering speed and system performance.

Algorithm 1: $O(n)$ – Linear Time Complexity

Asymptotic Analysis

- Big-O: $O(n)$
- Big-Theta: $\Theta(n)$
- Big-Omega: $\Omega(n)$

Explanation

The execution time increases directly in proportion to the input size. If the number of pixels or frames doubles, the processing time also approximately doubles.

Example:

- 1,000 pixels 1,000 operations
- 10,000 pixels 10,000 operations

Advantages

- Fast execution
- Low memory usage
- Excellent scalability
- Suitable for real-time rendering

Algorithm 2: $O(n \log n)$ – Linearithmic Time Complexity

Asymptotic Analysis

- Big-O: $O(n \log n)$
- Big-Theta: $\Theta(n \log n)$
- Big-Omega: $\Omega(n \log n)$

Explanation

The running time grows slightly faster than linear because of the logarithmic factor. It performs well for large datasets while handling additional operations such as sorting or hierarchical processing.

Example:

- 1,000 pixels $\approx 10,000$ operations
- 10,000 pixels $\approx 140,000$ operations

Advantages

- Good balance between speed and functionality
- Efficient for texture sorting and scene organization
- Scales well for complex rendering

Algorithm 3: $O(n^2)$ – Quadratic Time Complexity

Asymptotic Analysis

- Big-O: $O(n^2)$

- Big-Theta: $\Theta(n^2)$
- Big-Omega: $\Omega(n^2)$

Explanation

The running time increases with the square of the input size. A small increase in image resolution causes a very large increase in computation time.

Example:

- 1,000 pixels 1,000,000 operations
- 10,000 pixels 100,000,000 operations

Disadvantages

- Very slow for large media files
- High CPU/GPU utilization
- Poor scalability
- Unsuitable for real-time rendering

Performance for High-Resolution Media Files

When processing high-resolution media (4K, 8K, HDR, or VR):

- $O(n)$:
 - Processes each pixel or frame once.
 - Maintains fast rendering even for very large files.
 - Ideal for streaming and real-time graphics.
- $O(n \log n)$:
 - Adds a small overhead due to sorting or hierarchical processing.
 - Performance remains efficient for large multimedia datasets.
 - Suitable for advanced rendering engines.
- $O(n^2)$:
 - Every element interacts with many others, causing computation to increase dramatically.
 - Rendering becomes slow and may introduce delays or frame drops.
 - Not suitable for high-resolution multimedia applications.

Most Efficient Algorithm

The $O(n)$ algorithm is the most efficient for large-scale multimedia rendering.

Justification

- Lowest computational complexity.
- Processing time grows linearly with input size.
- Handles millions of pixels and frames efficiently.
- Consumes fewer CPU/GPU resources.
- Provides excellent scalability for 4K, 8K, VR, and cloud rendering.

- Supports real-time multimedia applications with minimal latency.

Although $O(n \log n)$ is also efficient for operations such as sorting textures or organizing rendering data, $O(n)$ delivers the best overall computational efficiency when every media element only needs to be processed once.

Conclusion

Asymptotic analysis shows that $O(n)$ has the slowest growth rate and offers the best scalability, making it the optimal choice for large-scale multimedia rendering. $O(n \log n)$ is a practical option when additional processing like sorting is required. $O(n^2)$ should be avoided for high-resolution media because its quadratic growth results in excessive execution time and poor performance as input size increases.

Q2. Social Media Feed Ranking

A social media platform ranks posts using: $T(n) = 4T(n/2) + n^2$. Apply the Master Theorem to compute the time complexity. Clearly specify parameters and case selection. Analyze scalability issues as the number of posts increases and interpret efficiency.

ANSWER:

Given the recurrence relation:

where:

- n = Number of social media posts
- $4T(n/2)$ = Problem is divided into 4 equal subproblems.
- n^2 = Time required to rank, merge, and process posts.

Step 1: Identify Master Theorem Parameters

The standard Master Theorem form is:

Compare with the given recurrence:

- $a = 4$
- $b = 2$

- $f(n) = n^2$

Step 2: Compute

Therefore,

Step 3: Compare with

Since both are equal,

Step 4: Select the Master Theorem Case

This satisfies Case 2 of the Master Theorem:

If

where $k = 0$, then

Hence,

Final Time Complexity

Scalability Analysis

As the number of posts increases:

- The factor causes computation to grow rapidly.
- The additional factor adds further overhead due to recursive divisions.
- For small datasets, ranking is reasonably fast.
- For very large social media platforms with millions of posts, execution time increases significantly, leading to higher CPU usage and slower feed generation.

Example:

Number of

Posts (n)

Approximate Growth

100

1,000

1,000,000

Extremely large
computation

This shows that processing time increases much faster than the number of posts.

Efficiency Interpretation

- Advantages:
 - o Recursive processing divides the ranking problem into smaller parts.
 - o Produces accurate and consistent ranking results.
- Limitations:
 - o Time complexity $\Theta(n^2 \log n)$ is relatively high.
 - o Not ideal for real-time ranking of massive social media feeds.
 - o Requires more processing power and memory as data grows.

Conclusion

Using the Master Theorem, the recurrence relation

falls under Case 2, giving the final complexity:

Although the recursive approach is systematic, its quadratic-logarithmic growth makes it less scalable for very large numbers of posts. For modern social media platforms handling millions of posts, more efficient algorithms with $O(n \log n)$ or $O(n)$ complexity are preferred to ensure faster feed ranking, lower computational cost, and better scalability.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
--------------	--	-------------------------------	--------------------------------	----------------------------	---------------	-------------

Q1

(50 Marks)

Q2

(50 Marks)

Overall Total (100)

Faculty In-charge

Course Coordinator

Head of Department

Signature:

Date:

Signature:

Date:

Signature:

Date: